

Mission 004: Live Truth Ingestion and Storytelling Mission Packet

This mission takes Agents-Vis beyond the production closeout by making the displayed timeline reflect the updated truth continuously, and by refactoring what agents write so the story shown in the app is clearer and more useful.

User goal

Keep the app read-only for users, but make the data behind it stay current through a real ingestion process, and make the mission timeline tell a clearer story instead of just listing details.

Core product decision

Keep the single live timeline experience. Do not add a second dashboard view. Improve the source pipeline and the writing contract together so the app shows the right truth in a clearer narrative form.

Scope in

- One primary chronological timeline for the latest mission only
- A live ingestion path that updates the canonical source of truth
- A production-safe source that can be refreshed continuously
- Stable read-only API contract
- Explicit freshness / stale / lag indicators
- Full-history replay remains visible and scrollable
- Parallel activity remains visibly parallel
- Refactor the way agents write updates so the story is clearer and more consistent
- Production smoke verification on the deployed URL after ingestion and story changes

Story contract

- Each update should communicate what changed, who changed it, why it matters, and what comes next if relevant
- Required narrative fields for readable updates: actorName, actorRole, action, timestamp, sequenceIndex
- detail should be present whenever possible
- Parallel work must stay in one timeline and be grouped with parallelGroupId, parallelOrder, and parallelSize
- Use plain language and outcome-oriented action phrases
- Avoid vague or filler wording like “did work”, “handled stuff”, or “TBD” in visible story text
- Do not expose raw JSON or internal schema labels in the visible story

Scope out

- Multi-mission browser
- Admin tools
- Mutation controls for end users
- Analytics dashboards
- Rewriting the timeline UX into a second summary view
- Requiring webhook if another ingestion method is safer or simpler

Team handoff

- Backend owns the live ingestion path, canonical source, API stability, and freshness behavior
- Frontend owns the story presentation, readability, and live-state rendering
- Product owns the writing contract that agents should follow so the timeline tells a better story
- QA owns production smoke tests, ordering checks, freshness checks, and release verification
- Coordinator owns sequencing, blockers, release gates, and status updates
- Every product, backend, frontend, and QA step must be documented as part of the mission output
- The authoritative mission documentation must be saved in the repo as PDF files alongside the mission docs
- Each role handoff should name the documentation artifact it produced or updated

Acceptance criteria

- The displayed timeline is fed by an updated source of truth, not a frozen snapshot
- The team can explain how data gets from the source into the app
- The app still shows only the latest mission timeline
- The timeline stays chronological and preserves parallel activity
- Freshness / lag / stale states are visible in production
- The story in the timeline is easier to follow because the agent-written entries are better structured
- The API contract remains read-only and stable
- Production smoke checks pass on the live deployment

Open questions

- What is the canonical source of truth in Mission 004: external event store, production database, or another feed?
- Which ingestion method is safest and simplest for that source: webhook, polling, scheduled sync, or another path?
- What writing contract should agents follow so the story reads clearly?
- What fields must be present in every agent update to support the timeline narrative?
- What smoke checks should be mandatory before release is considered done?

Dependencies / blockers

- External source access or production database access, if the canonical source is outside the repo
- Agreement on the agent writing contract before the refactor starts
- Deployment access and production smoke verification ability

Notes

- Webhook is optional, not required
- The source-of-truth pipeline and the story-writing refactor should be treated as linked work, not separate product ideas
- Keep the UI read-only and consumer-friendly
- Do not add a second dashboard view